

DD: BIL-01-PAY-01 — Payment Application

Developer implementation guide: DD_BIL-01-PAY-01-Payment_Application-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract
DD_BIL-01-PAY-01-Payment_Application-v1.0.md.

Verdict: ■ New | **Wave:** 1 | **Group II:** Billing | **Version:** v1.0 Satellite of **BIL-01 Charging Engine (Core)**. Owns inbound money movement: receiving payments from gateway adapters (M-Pesa, Visa, bank transfer), from STA-01's mark-paid-offline operation, and from any other source; allocating each payment across the account's outstanding invoices (POSTPAID) or crediting the account wallet (PREPAID); handling overpayments via configurable policy; supporting payment reversal. Emits the PaymentApplied events that BIL-02-STA-01 consumes to transition invoice status. **Parent DD:** BIL-01-Charging_Engine-Core. Shares the billing-charging module deployment, the settlement adapters (to BIL-02-GEN-01 / BIL-05), and the bil_01_config deployment config. PAY-01 owns its own ledger tables (payment_ledger, payment_invoice_allocation, account_credit_balance) and entry points (/api/payments/*). **Version history:** see CHANGELOG_BIL-01-PAY-01-Payment_Application.md in the same directory.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/05_billing/DD_BIL-01-PAY-01-Payment_Application-v1.0.md
Version	v1.0
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- GET /api/credit-balance/{accountId}
- GET /api/payments/{id}
- GET /api/payments/{paymentId}
- GET /api/payments?accountId=acc_01HX_HOME&from=2026-05-01&to=2026-05-31&limit=20

- POST /api/admin/credit-balance/cust_01HX.../adjust
- POST /api/admin/credit-balance/{accountId}/apply
- POST /api/admin/credit-balance/{account_id}/adjust
- POST /api/admin/payments/pmt_01HZ...C/allocate-surplus
- POST /api/admin/payments/{id}/allocate-surplus
- POST /api/payments/pmt_01HZ...D/reverse
- POST /api/payments/receive
- POST /api/payments/receive,
- POST /api/payments/{paymentId}/reverse
- POST /api/wallets/credit
- POST /api/wallets/debit

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- billing-charging
- mark-paid-offline

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- InvoiceIssued
- PaymentApplied
- PaymentReceivedUnallocated
- PaymentReversed

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.

4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group II:** Billing | **Version:** v1.0

Satellite of **BIL-01 Charging Engine (Core)**. Owns inbound money movement: receiving payments from gateway adapters (M-Pesa, Visa, bank transfer), from STA-01's mark-paid-offline operation, and from any other source; allocating each payment across the account's outstanding invoices (POSTPAID) or crediting the account wallet (PREPAID); handling overpayments via configurable policy; supporting payment reversal. Emits the `PaymentApplied` events that BIL-02-STA-01 consumes to transition invoice status.

Parent DD: BIL-01-Charging_Engine-Core. Shares the billing-charging module deployment, the settlement adapters (to BIL-02-GEN-01 / BIL-05), and the `bil_01_config` deployment config. PAY-01 owns its own ledger tables (`payment_ledger`, `payment_invoice_allocation`, `account_credit_balance`) and entry points (`/api/payments/*`).

Version history: see `CHANGELOG_BIL-01-PAY-01-Payment_Application.md` in the same directory.

Glossary

Term	Meaning
Payment receipt	The act of recording that money came in from a customer (or on a customer's behalf). Payments arrive from payment gateway adapters (M-Pesa, Visa, bank transfer integrations) calling <code>POST /api/payments/receive</code> , from STA-01's mark-paid-offline operation, or from BIL-03 Cycle Close for auto-payment scenarios.
Payment allocation	Distributing a received payment's amount across the account's outstanding invoices (POSTPAID) or crediting the wallet (PREPAID). Owned by this DD.
Customer-directed allocation	The request includes <code>target_invoice_id</code> , telling PAY-01 which invoice to credit.
Auto allocation	No target invoice provided; PAY-01 picks invoices for the same account per the operator's allocation policy (<code>FIFO_DUE_DATE</code> / <code>FIFO_ISSUE_DATE</code> / <code>LARGEST_FIRST</code> / <code>SMALLEST_FIRST</code>).

Term	Meaning
Overpayment	A POSTPAID payment whose <code>paid_amount</code> exceeds the sum of the account's open invoice outstanding amounts. The surplus is handled per the operator's overpayment policy.
Account credit balance	A per-account running balance of surplus money waiting to be applied to future invoices. Stored in <code>account_credit_balance</code> . Drawn from overpayments and (per CN-01) from credit note surplus.
Payment reversal	Undoing a previously-applied payment (e.g., gateway chargeback, disputed payment). Reverses the allocations, restores invoice outstanding amounts, debits the wallet if PREPAID, emits <code>PaymentReversed</code> .
Unallocated payment	A payment received with no allocation target — customer has no open invoices and no PREPAID wallet, or operator's overpayment policy is <code>MANUAL_REVIEW</code> . Recorded in <code>payment_ledger</code> with <code>status = UNALLOCATED</code> ; emits <code>PaymentReceivedUnallocated</code> for admin attention.

A — Target

1. What this process is about

When a customer pays — through any channel — the money needs to be recorded, allocated to invoices (POSTPAID) or wallet (PREPAID), and the resulting status changes propagated. PAY-01 owns this flow end-to-end:

Receipt: gateway adapters or admin tools call `POST /api/payments/receive` with the payment details (account, customer identity reference, amount, currency, method, reference, optional target invoice). PAY-01 deduplicates on (`account_id`, `payment_reference`) — gateways may retry the same notification.

Allocation: PAY-01 determines the customer's billing mode. PREPAID → credit the wallet via BIL-05. POSTPAID → allocate across open invoices, either to the customer-specified target invoice or per the operator's auto-allocation policy (FIFO by due date, by issue date, by largest amount, or by smallest amount).

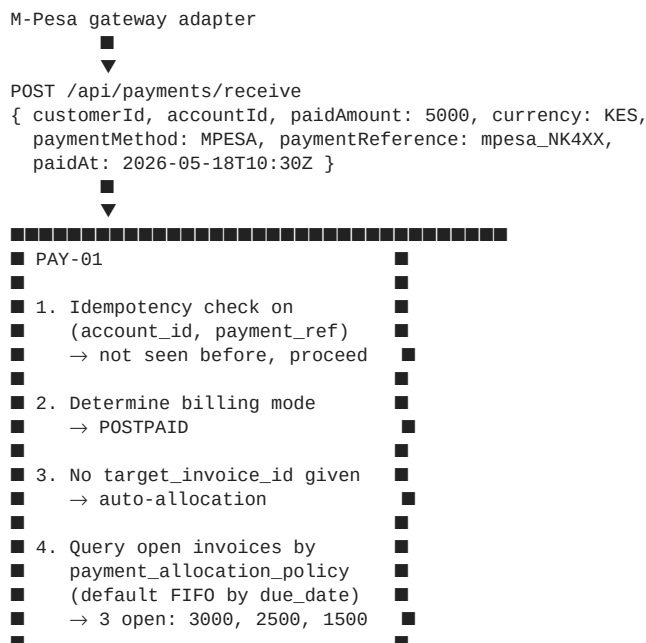
Overpayment: if a POSTPAID payment exceeds total outstanding, the surplus goes to: (a) `account_credit_balance` for use against future invoices on the same account (default), (b) a designated overflow wallet, or (c) held for admin manual review.

Audit: every payment writes to `payment_ledger`; every allocation against an invoice writes to `payment_invoice_allocation`. All in one DB transaction.

Downstream notification: PAY-01 emits one `PaymentApplied` event per allocation. STA-01 consumes these to transition invoice status (PENDING → PAID / PARTIALLY_PAID; OVERDUE → PAID / PARTIALLY_PAID; etc.).

Reversal: admin or gateway-driven reversal undoes allocations and restores outstanding amounts. Emits `PaymentReversed`.

Flow overview for a POSTPAID gateway payment



```
STA-01 consumes PaymentApplied events
- Invoice #1: PENDING → PAID
- Invoice #2: OVERDUE → PARTIALLY_PAID
```

What PAY-01 explicitly does not own

- **Charging** (outbound) — owned by **BIL-01 Core**. A charge debits the customer; a payment credits. Different flows, different ledgers, different entry points.
- **Credit / debit note application** — owned by **BIL-01-CN-01**. Notes arrive via Kafka events from GEN-01; payments arrive via REST or other gateways.
- **Payment gateway integration** — owned by adapter DDs (M-Pesa adapter, Visa adapter, bank transfer adapter). Adapters call PAY-01's REST; the wire-level integration with each payment provider is the adapter's responsibility.
- **Invoice status transitions** — owned by **BIL-02-STA-01**. PAY-01 emits `PaymentApplied`; STA-01 consumes and transitions.
- **Wallet balance / top-up mechanics** — owned by **BIL-05**. PAY-01 calls BIL-05 for wallet credits.
- **Invoice generation** — owned by **BIL-02-GEN-01**. PAY-01 reads `invoice.outstanding_amount` for allocation decisions.
- **Refunds initiated by operator policy** (different from gateway chargebacks) — typically handled via BIL-02-ADJ-01 credit note workflow, which feeds back through CN-01.

2. Business rules

Rules grouped: RC (receipt), AL (allocation), OV (overpayment), RV (reversal), EV (event emission).

Rule group RC — Receipt

ID	Rule	Triggered on
R-PAY-01-RC-1	Payment events arrive on three entry points: POST /api/payments/receive (from gateway adapters and from STA-01's mark-paid-offline), Kafka consumer for <code>sophix.payments.gateway.received</code> (asynchronous gateway notifications), and synchronous internal calls (from BIL-03 Cycle Close for auto-payment scenarios). All three funnel into the same allocation pipeline.	Every payment receipt
R-PAY-01-RC-2	A payment carries: <code>account_id</code> , <code>customer_id</code> (denormalized), <code>operator_code</code> , <code>paid_amount</code> , <code>currency</code> , <code>payment_method</code> (MPESA / VISA / BANK_TRANSFER / CASH / OFFLINE_RECORDED / etc.), <code>payment_reference</code> (gateway-provided ID, used as idempotency key), <code>paid_at</code> , optional <code>target_invoice_id</code> (account-directed), optional actor (admin user ID when recorded via mark-paid-offline). The <code>payment_method</code> is informational; allocation logic is identical across methods.	Every receipt

ID	Rule	Triggered on
R-PAY-01-RC-3	Idempotency uses (account_id, payment_reference) tuple. If a payment with the same reference has already been applied successfully (status IN ('APPLIED', 'PARTIALLY_APPLIED', 'UNALLOCATED')), PAY-01 returns the prior result without re-applying. Gateway adapters may retry the same notification multiple times; this deduplicates. Reversed payments do NOT block a new application with the same reference (rare but possible — e.g., reversal due to chargeback, then the account pays again with the same gateway txid).	Every receipt
R-PAY-01-RC-4	The account's billing mode is resolved from the target invoice, target wallet, or SUB-LM subscription/account context at receipt time. PREPAID → wallet path (rule R-PAY-01-AL-3). POSTPAID → invoice allocation path. Accounts with mixed-mode subscriptions require explicit target_invoice_id if the payment is intended for a POSTPAID invoice, OR explicit target_wallet_ref for PREPAID wallet top-up. Without target, the payment is rejected with ACCOUNT_MIXED_MODE_REQUIRES_TARGET.	Every receipt
R-PAY-01-RC-5	Currency consistency: the payment currency MUST match the account's billing currency (resolved via BIL-05's account-wallet mapping for PREPAID, via SUB-LM/account context for POSTPAID). Mismatch is rejected with PAYMENT_CURRENCY_MISMATCH. Multi-currency handling is out of scope.	Every receipt

Rule group AL — Allocation

ID	Rule	Triggered on
R-PAY-01-AL-1	POSTPAID allocation — customer-directed. When target_invoice_id is provided, the payment is applied entirely to that invoice. If paid_amount exceeds the invoice's outstanding_amount, the surplus follows the operator's overflow policy (rule group OV). If paid_amount is less than outstanding_amount, the invoice receives a partial payment (the resulting status change PARTIALLY_PAID is owned by STA-01).	POSTPAID payment with target_invoice_id
R-PAY-01-AL-2	POSTPAID allocation — auto. When target_invoice_id is NOT provided, PAY-01 allocates across the account's open invoices using the operator's payment_allocation_policy: FIFO_DUE_DATE (default — oldest due date first), FIFO_ISSUE_DATE, LARGEST_FIRST, or SMALLEST_FIRST. The policy is stored in pay_01_config. For each invoice in policy order: apply min(paid_amount_remaining, outstanding_amount). Stop when paid_amount_remaining = 0 or no more open invoices.	POSTPAID payment without target_invoice_id
R-PAY-01-AL-3	PREPAID payment. PAY-01 calls POST /api/wallets/credit on BIL-05 with {account_id, customer_id, wallet_ref, amount, currency, payment_reference, idempotency_key}. BIL-05 credits the wallet and returns the new balance. The wallet_ref to credit is determined by: (a) explicit wallet_ref in the request if provided, (b) the account's default wallet otherwise (resolved via BIL-05).	PREPAID payment
R-PAY-01-AL-4	Open invoices for allocation are those with status IN ('PENDING', 'PARTIALLY_PAID', 'OVERDUE', 'OVERDUE_PARTIAL'). Invoices in DISPUTED are EXCLUDED from auto-allocation (the disputed amount is in question; applying payment to a disputed invoice would be premature). If a customer has ONLY DISPUTED invoices, auto-allocation produces zero allocations, and the payment proceeds to overpayment handling.	Auto allocation

ID	Rule	Triggered on
R-PAY-01-AL-5	Each allocation against a specific invoice writes a <code>payment_invoice_allocation</code> row recording: <code>payment_id</code> , <code>invoice_id</code> , <code>allocated_amount</code> , <code>outstanding_before</code> , <code>outstanding_after</code> , <code>allocation_strategy</code> . Atomicity: all allocations for one payment commit in one DB transaction together with the parent <code>payment_ledger</code> row.	Every allocation

Rule group OV — Overpayment

ID	Rule	Triggered on
R-PAY-01-OV-1	When a POSTPAID payment leaves <code>paid_amount_remaining > 0</code> after allocation, surplus handling follows operator config (<code>overpayment_policy</code> in <code>pay_01_config</code>): <code>APPLY_TO_NEXT_OPEN</code> (default), <code>WALLET_TOPUP</code> , <code>MANUAL_REVIEW</code> .	Every POSTPAID overpayment
R-PAY-01-OV-2	<code>APPLY_TO_NEXT_OPEN</code> (default): surplus is added to the account's <code>account_credit_balance</code> . Subsequent invoices (issued by GEN-01) trigger an auto-application: PAY-01 listens for <code>InvoiceIssued</code> events; if the account has a positive credit balance, PAY-01 applies it as a synthetic payment with <code>payment_method = CREDIT_BALANCE_APPLICATION</code> .	Overpayment with <code>APPLY_TO_NEXT_OPEN</code>
R-PAY-01-OV-3	<code>WALLET_TOPUP</code> : surplus credits a designated overflow wallet via BIL-05. The wallet to use is in <code>pay_01_config.overflow_wallet_ref</code> ; if no such wallet is configured, fall back to <code>APPLY_TO_NEXT_OPEN</code> .	Overpayment with <code>WALLET_TOPUP</code>
R-PAY-01-OV-4	<code>MANUAL_REVIEW</code> : surplus is held; the parent payment is marked <code>PARTIALLY_APPLIED</code> (or <code>UNALLOCATED</code> if nothing was allocated) and an <code>OverpaymentPendingReview</code> event is emitted. Admin must manually decide via <code>POST /api/admin/payments/{id}/allocate-surplus</code> . The money is NOT returned to the gateway automatically.	Overpayment with <code>MANUAL_REVIEW</code>
R-PAY-01-OV-5	An unallocated payment (no open invoices, no wallet to credit) is recorded with <code>status = UNALLOCATED</code> and emits <code>PaymentReceivedUnallocated</code> for admin attention. The payment is NOT rejected — the money is real and must be accounted for.	Payment with zero allocation targets
R-PAY-01-OV-6	<code>account_credit_balance</code> is mutable. Movements: increases from overpayment surplus and from CN-01's credit note surplus (CN-01 calls into PAY-01 for surplus-to-balance flow); decreases when PAY-01 applies it as a synthetic payment on a new invoice for the same account; admin can also debit it via <code>POST /api/admin/credit-balance/{account_id}/adjust</code> with mandatory reason for accounting corrections.	Credit balance updates

Rule group RV — Reversal

ID	Rule	Triggered on
R-PAY-01-RV-1	Payment reversal endpoint: <code>POST /api/payments/{paymentId}/reverse</code> with reason code and free-text justification. Reverses all <code>payment_invoice_allocation</code> rows for this payment: each invoice's <code>outstanding_amount</code> is restored by the previously-allocated amount. Wallet credits are reversed via <code>POST /api/wallets/debit</code> on BIL-05; if the wallet has insufficient balance to debit (customer already spent the topped-up amount), the reversal fails with <code>WALLET_INSUFFICIENT_FOR_REVERSAL</code> — admin must reconcile manually.	Payment reversal

ID	Rule	Triggered on
R-PAY-01-RV-2	Reversal writes a NEW row to payment_ledger with status = REVERSED, reversal_of_payment_id pointing to the original, reversal_reason_code, reversed_by. The original payment row's status is also updated to REVERSED. Both atomically in one transaction.	Every reversal
R-PAY-01-RV-3	Reversal emits one PaymentReversed event capturing the customer, the reversed amount, the reversed allocations array ([{invoice_id, amount}]), the reversal reason. STA-01 consumes and transitions affected invoices back (e.g., PAID → OVERDUE if reversal makes them past-due again, PAID → PARTIALLY_PAID if only part was reversed).	Per reversal
R-PAY-01-RV-4	Surplus that went to account_credit_balance is debited on reversal. If the balance has already been consumed by intervening invoices for the same account, the reversal proceeds best-effort (debts what's available) and emits PartialReversalDueToConsumedBalance for admin visibility.	Reversal of overpayment
R-PAY-01-RV-5	Reversal requires PAYMENT_REVERSAL role (defined in FOUNDATION_AUTH.md).	Authorization
R-PAY-01-RV-6	Reversal time-window enforcement is operator-configurable. Default: no time limit. Some gateways have hard windows (Visa 180 days, M-Pesa 14 days for reversal eligibility); operators implementing chargeback reconciliation should add their own gating before calling the reversal endpoint. PAY-01 does not enforce gateway-specific windows.	Reversal gating

Rule group EV — Event emission

ID	Rule	Triggered on
R-PAY-01-EV-1	PAY-01 emits the following Kafka events with defined schemas (envelope: eventType, aggregateId, timestamp, payload). Topics: sophix.billing.payment.applied, sophix.billing.payment.received-unallocated, sophix.billing.payment.reversed, sophix.billing.overpayment.pending-review, sophix.billing.credit-balance.adjusted.	Various
R-PAY-01-EV-2	PaymentApplied payload: payment_id, payment_reference, customer_id, operator_code, target (one of {type: INVOICE, invoice_id, applied_amount, outstanding_after} for POSTPAID; {type: WALLET, wallet_ref, applied_amount, balance_after} for PREPAID; or {type: CREDIT_BALANCE, applied_amount, balance_after} for surplus credited to customer balance), total_paid_amount, currency, payment_method, paid_at, applied_at. Emitted PER allocation — one payment allocated across N invoices produces N events. STA-01 consumes INVOICE-target events for status transitions; BIL-05 audit consumes WALLET-target events; analytics consumes all.	Per allocation
R-PAY-01-EV-3	PaymentReceivedUnallocated payload: payment_id, customer_id, operator_code, paid_amount, currency, payment_method, payment_reference, received_at, reason (NO_OPEN_INVOICES_NO_WALLET / ALL_INVOICES_DISPUTED / MIXED_MODE_NO_TARGET / MANUAL_REVIEW_POLICY). Consumed by admin alert systems and BIL-04 Collections.	Per unallocated payment

ID	Rule	Triggered on
R-PAY-01-EV-4	OverpaymentPendingReview payload: payment_id, customer_id, operator_code, surplus_amount, currency, received_at. Emitted only when overpayment_policy = MANUAL_REVIEW. Admin must manually allocate or refund the surplus.	Per MANUAL_REVIEW overpayment
R-PAY-01-EV-5	PaymentReversed payload: payment_id (the reversal row's ULID), original_payment_id, customer_id, operator_code, reversed_amount, reversed_allocations (array of {invoice_id, amount}), currency, reversed_at, reversal_reason_code, reversed_by. STA-01 consumes to transition affected invoices.	Per reversal
R-PAY-01-EV-6	CreditBalanceAdjusted payload: customer_id, operator_code, direction (CREDIT / DEBIT), amount, balance_before, balance_after, reason_code, triggered_by (PAYMENT_OVERPAYMENT_SURPLUS / CREDIT_NOTE_SURPLUS / INVOICE_APPLICATION / ADMIN_ADJUSTMENT), adjusted_at. Consumed by analytics and finance ETL.	Per credit_balance change

3. Module owner and impacted modules

Role	Module	What it does
Owner	billing-charging module (shared with BIL-01 Core and BIL-01-CN-01), sub-package payment	Owns payment_ledger, payment_invoice_allocation, account_credit_balance, pay_01_config tables. REST endpoints /api/payments/*. Kafka consumer for sophix.payments.gateway.received. Kafka consumer for InvoiceIssued (to trigger credit-balance auto-application).
Impacted (callees)	BIL-02-Core (read invoice.outstanding_amount), BIL-02-GEN-01 (no direct call; PAY-01 may listen for InvoiceIssued to apply credit balance), BIL-05 (wallet credit / debit for PREPAID payments, overflow wallets, reversal)	Receive synchronous REST or read calls.
Impacted (callers)	Payment gateway adapters (M-Pesa, Visa, bank transfer), STA-01 (mark-paid-offline), BIL-03 Cycle Close (auto-payment), backoffice admin UI (manual receipt + reversal + surplus allocation + credit balance adjustment)	Issue payment receipt / reversal requests.
Impacted (downstream consumers)	BIL-02-STA-01 (consumes PaymentApplied / PaymentReversed to transition invoice status), BIL-04 Collections (consumes PaymentReceivedUnallocated), DD_NOT-01 Notification Service (consumes PaymentApplied for customer confirmation), CVM, analytics, finance ETL	Consume PAY-01 emission events.
Engines	Kafka (consumers + producers). No Drools, no Camunda in PAY-01 — allocation logic is straightforward enough to keep in code.	—

4. Foundation references

DB → FOUNDATION_DB.md · Kafka → FOUNDATION_KAFKA.md · Cache → FOUNDATION_CACHE.md (Redis for idempotency keys) · Auth → FOUNDATION_AUTH.md (introduces PAYMENT_GATEWAY_INTERNAL, PAYMENT_REVERSAL, CREDIT_BALANCE_ADJUSTER roles).

Technical design

Code structure

modules/billing-charging/

```

src/main/java/com/sophix/billing/charging/payment/
service/
    PaymentReceiptService.java      ← entry-point handler: dedup, route to allocator
    PaymentAllocator.java          ← billing-mode dispatch, postpaid auto/directed, prepaid
    PostpaidAllocator.java          ← invoice allocation strategies
    OverpaymentHandler.java         ← APPLY_TO_NEXT_OPEN / WALLET_TOPUP / MANUAL_REVIEW
    CreditBalanceService.java       ← account_credit_balance read/write, auto-apply on InvoiceIssued
    PaymentReversalService.java     ← reverse allocations, restore outstanding, debit wallet
    PaymentLedgerWriter.java
kafka/
    GatewayPaymentEventConsumer.java ← consumes sophix.payments.gateway.received
    InvoiceIssuedConsumer.java       ← drives credit-balance auto-application
rest/
    PaymentResource.java            ← POST /api/payments/receive, /reverse, GET /api/payments/{id}
    AdminPaymentResource.java       ← surplus allocation, credit balance adjustment
domain/
    Payment.java
    PaymentAllocation.java
    AllocationStrategy.java         ← enum
    OverpaymentPolicy.java         ← enum
    AccountCreditBalance.java
    PaymentStatus.java
repository/
    PaymentLedgerRepository.java
    PaymentAllocationRepository.java
    AccountCreditBalanceRepository.java
    Pay01ConfigRepository.java
event/
    PaymentAppliedEventPublisher.java
    PaymentReversedEventPublisher.java
    PaymentReceivedUnallocatedEventPublisher.java
    OverpaymentPendingReviewEventPublisher.java
    CreditBalanceAdjustedEventPublisher.java
config/
    Pay01Config.java
src/main/resources/
    config/liquibase/changelog/billing-charging/
        010_payment_ledger.xml
        011_payment_invoice_allocation.xml
        012_account_credit_balance.xml
        013_pay_01_config.xml

```

Data model

payment_ledger

Append-only ledger of every payment received. Partitioned by month.

Column	Type	Notes
payment_id	TEXT PK	ULID, server-generated. Example: pmt_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
account_id	TEXT NOT NULL	Operational billing account receiving the payment. Example: acc_01HX_HOME
customer_id	TEXT NOT NULL	Denormalized identity reference for search/audit. Example: cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6
operator_code	TEXT NOT NULL	Operator tenant. Values: WIK (Wananchi KE), WUG (Wananchi UG), WTZ (Wananchi TZ), YASSN (Yas Senegal)
payment_method	TEXT NOT NULL	Enum. Values: MPESA (M-Pesa mobile money) / VISA (card via gateway) / BANK_TRANSFER (direct deposit) / CASH (in-person, recorded by agent) / OFFLINE_RECORDED (admin mark-paid-offline via STA-01) / CREDIT_BALANCE_APPLICATION (synthetic payment from customer's credit balance on new invoice)
payment_reference	TEXT NOT NULL	Gateway-provided reference; used as idempotency key with account_id. Examples: mpesa_NK4XX7QP9 (M-Pesa transaction ID), visa_ch_abc123 (Stripe-style charge ID), manual_ref_2026051801 (admin-entered when no gateway involved)
paid_amount	NUMERIC(18,2) NOT NULL	The total amount received. Example: 6402.50
currency	TEXT NOT NULL	ISO 4217. Example: KES

Column	Type	Notes
paid_at	TIMESTAMP NOT NULL	When the customer made the payment, gateway-reported. Example: 2026-05-18 10:30:00
received_at	TIMESTAMP NOT NULL DEFAULT NOW()	When PAY-01 received the notification (may differ from paid_at by minutes/hours for async gateways). Example: 2026-05-18 10:30:05
status	TEXT NOT NULL	Enum. Values: APPLIED (fully consumed by invoice allocations or wallet credit) / PARTIALLY_APPLIED (some allocated, some held as customer credit balance or pending review) / UNALLOCATED (no allocation target; money real but unplaced) / REVERSED (this row is either a reversal record or was reversed by a later row)
target_invoice_id_hint	TEXT NULL	Customer-directed allocation hint from the receive request. NULL when auto-allocation. Example: inv_01HZX5M2K9P3Q7R4T6W8Y0Z2N4
total_allocated_amount	NUMERIC(18,2) NOT NULL DEFAULT 0	Sum across payment_invoice_allocation rows. Example: 6402.50
total_credited_to_balance	NUMERIC(18,2) NOT NULL DEFAULT 0	Surplus credited to account_credit_balance (POSTPAID overpayment with APPLY_TO_NEXT_OPEN). Example: 500.00
total_credited_to_wallet	NUMERIC(18,2) NOT NULL DEFAULT 0	Amount credited to a PREPAID wallet (full PREPAID payment, or POSTPAID overpayment with WALLET_T0PUP policy). Example: 3000.00
reversal_of_payment_id	TEXT NULL	If this row's status = REVERSED AND it IS a reversal record, points to the original payment row. NULL on the original. Example: pmt_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
reversal_reason_code	TEXT NULL	Enum. Values: GATEWAY_CHARGEBACK (M-Pesa/Visa reversed on their side) / DISPUTED_PAYMENT (customer disputed; ADJ-01 resolved against operator) / DUPLICATE_PAYMENT (gateway double-charged) / ADMIN_CORRECTION (admin error correction)
reversed_by	TEXT NULL	Admin user ID who triggered the reversal. Example: admin_user_42
reversed_at	TIMESTAMP NULL	Example: 2026-05-18 14:00:00
actor	TEXT NULL	If admin-initiated (mark-paid-offline, reversal). Example: admin_user_17. NULL for gateway-driven payments
request_payload	JSONB NOT NULL	Sanitized snapshot of the receive request. Example: {"customerId": "cust_01HX...", "accountId": "acc_01HX_HOME", "operatorCode": "WIK", "paidAmount": 6402.50, "currency": "KES", "paymentMethod": "MPESA", "paymentReference": "mpesa_NK4XX...", "paidAt": "2026-05-18T10:30:00Z", "targetInvoiceId": null}

Indexes: PK; UNIQUE on (account_id, payment_reference) (idempotency); (account_id, paid_at DESC), (customer_id, paid_at DESC), (operator_code, status, paid_at DESC), (reversal_of_payment_id) WHERE reversal_of_payment_id IS NOT NULL. Monthly partitions. P10Y retention.

Sample data:

payment_id	account	customer	payment_method	paid_amount	status	total_allocated	total_to_balance	total_to_wallet
pmt_01HZ...A	acc_01HX_HOME	cust_01HX..	MPESA	6402.50	APPLIED	6402.50	0.00	0.00
pmt_01HZ...B	acc_01HX_HOME	cust_01HX..	VISA	5000.00	PARTIALLY_APPLIED	4500.00	500.00	0.00
pmt_01HZ...C	acc_01HX_SHOP	cust_01HX..	OFFLINE_RECORD	2000.00	UNALLOCATED	0.00	0.00	0.00
pmt_01HZ...D	acc_01HY_HOME	cust_01HY..	MPESA	1500.00	REVERSED	0.00	0.00	0.00
pmt_01HZ...E	acc_01HY_HOME	cust_01HY..	MPESA	3000.00	APPLIED	0.00	0.00	3000.00

payment_id	account	customer	payment_method	paid_amount	status	total_allocated	total_to_balance	total_to_wallet
pmt_01HZ...F	acc_01HX_HOME	cust_01HX...	CREDIT_BALANCE_APPLICATION	500.00	APPLIED	500.00	0.00	0.00

How to read: Row A — clean M-Pesa POSTPAID payment fully consumed by outstanding invoices. Row B — 5000 Visa POSTPAID partially allocated; 500 surplus went to credit balance (operator policy APPLY_TO_NEXT_OPEN). Row C — admin recorded an offline cash payment but no allocation target existed (customer had no open invoices, customer is mixed-mode without a target). Row D — a previously-applied payment was reversed. Row E — PREPAID customer M-Pesa payment credited entirely to wallet (no invoice allocation since the customer is PREPAID). Row F — a synthetic payment auto-applied from the customer's credit balance when a new invoice was issued.

payment_invoice_allocation

One row per (payment, invoice) allocation.

Column	Type	Notes
id	TEXT PK	ULID. Example: alloc_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
payment_id	TEXT NOT NULL	FK to payment_ledger.payment_id. Example: pmt_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
invoice_id	TEXT NOT NULL	FK to invoice.invoice_id in BIL-02 Core. Example: inv_01HZX5M2K9P3Q7R4T6W8Y0Z2N4
account_id	TEXT NOT NULL	Operational billing account for this allocation. Example: acc_01HX_HOME
customer_id	TEXT NOT NULL	Denormalized identity reference. Example: cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6
operator_code	TEXT NOT NULL	Denormalized. Values: WIK / WUG / WTZ / YASSN
allocated_amount	NUMERIC(18,2) NOT NULL	The portion of the parent payment applied to this specific invoice. Example: 3000.00
invoice_outstanding_before	NUMERIC(18,2) NOT NULL	The invoice's outstanding_amount BEFORE this allocation. Example: 3000.00
invoice_outstanding_after	NUMERIC(18,2) NOT NULL	AFTER this allocation. Example: 0.00 (fully paid) or 1000.00 (partial)
allocation_strategy	TEXT NOT NULL	Enum recording how this allocation was decided. Values: CUSTOMER_DIRECTED (request had target_invoice_id) / AUTO_FIFO_DUE_DATE / AUTO_FIFO_ISSUE_DATE / AUTO_LARGEST_FIRST / AUTO_SMALLEST_FIRST (auto per operator policy) / OVERFLOW_FROM_TARGET (overflow from a customer-directed target into next invoices) / CREDIT_BALANCE_AUTO_APPLY (synthetic from credit balance on new invoice)
allocated_at	TIMESTAMP NOT NULL DEFAULT NOW()	Example: 2026-05-18 10:30:05
reversed_at	TIMESTAMP NULL	If parent payment was reversed, the time this allocation was undone. Example: 2026-05-18 14:00:00

Indexes: PK; (payment_id), (invoice_id, allocated_at DESC), (account_id, allocated_at DESC), (customer_id, allocated_at DESC).

Sample data:

id	payment_id	invoice_id	allocated_amount	outstanding_before	outstanding_after	strategy
alloc_01HZ...1	pmt_01HZ...A	inv_01HZ...X	6402.50	6402.50	0.00	CUSTOMER_DIRECTED
alloc_01HZ...2	pmt_01HZ...B	inv_01HZ...Y	3000.00	3000.00	0.00	AUTO_FIFO_DUE_DATE
alloc_01HZ...3	pmt_01HZ...B	inv_01HZ...Z	1500.00	2500.00	1000.00	AUTO_FIFO_DUE_DATE
alloc_01HZ...4	pmt_01HZ...F	inv_01HZ...NEW	500.00	1200.00	700.00	CREDIT_BALANCE_AUTO_APPLY

How to read: Row 1 — single payment fully consumed by one invoice (customer-directed; one allocation row). Rows 2 + 3 — one payment split across two invoices using FIFO-by-due-date: older invoice fully paid (3000), next partially paid (1500 of 2500). Row 4 — credit balance synthetic payment auto-applied when a new invoice was issued; reduced outstanding from 1200 to 700.

account_credit_balance

Column	Type	Notes
account_id	TEXT PK	One row per operational account per operator. Example: acc_01HX_HOME
customer_id	TEXT NOT NULL	Denormalized identity reference for search/audit. Example: cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6
operator_code	TEXT NOT NULL	Values: WIK / WUG / WTZ / YASSN
current_balance	NUMERIC(18,2) NOT NULL DEFAULT 0	Current unallocated credit available for future invoices. Example: 500.00
currency	TEXT NOT NULL	ISO 4217. Example: KES
last_updated_at	TIMESTAMP NOT NULL	Time of last credit/debit/auto-apply. Example: 2026-05-18 12:00:00

Indexes: PK; (operator_code), (operator_code, current_balance) WHERE current_balance > 0 for finance reporting.

Sample data:

account_id	customer_id	current_balance	currency	last_updated_at
acc_01HX_HOME	cust_01HX...	500.00	KES	2026-05-18 12:00:00
acc_01HY_HOME	cust_01HY...	0.00	KES	2026-05-18 09:30:00
acc_01HZ_SHOP	cust_01HZ...	1200.00	KES	2026-05-15 18:45:00

How to read: First customer has 500 KES of unallocated credit (surplus from earlier overpayment that has not been consumed by a new invoice yet). Second customer's balance is zero (typical). Third customer holds 1200 KES — hasn't been issued a new invoice since the credit was created, or operator policy has not auto-applied. Movement history is in payment_ledger (when balance was credited/debited) joined to payment_invoice_allocation via strategy CREDIT_BALANCE_AUTO_APPLY.

pay_01_config

Per-operator (one row per operator deployment).

Column	Type	Notes
operator_code	TEXT PK	Values: WIK / WUG / WTZ / YASSN
payment_allocation_policy	TEXT NOT NULL DEFAULT 'FIFO_DUE_DATE'	Enum. Values: FIFO_DUE_DATE (oldest due date first — default; standard collections practice) / FIFO_ISSUE_DATE (oldest invoice first by issuance) / LARGEST_FIRST (largest outstanding first — useful when reducing aging on big balances matters more than chronological order) / SMALLEST_FIRST (smallest first — quickly clears short invoices)
overpayment_policy	TEXT NOT NULL DEFAULT 'APPLY_TO_NEXT_OPEN'	Enum. Values: APPLY_TO_NEXT_OPEN (surplus → account_credit_balance; auto-applied on next invoice issuance for the same account) / WALLET_TOPUP (surplus → designated overflow wallet via BIL-05) / MANUAL_REVIEW (surplus held for admin to decide)
overpayment_overflow_wallet_ref	TEXT NULL	Required if overpayment_policy = WALLET_TOPUP; otherwise NULL. References a wallet in BIL-05. Example: wal_default_topup_WUG
enable_credit_balance_auto_apply	BOOLEAN NOT NULL DEFAULT TRUE	If false, credit balance is NEVER auto-applied to new invoices; admin must explicitly trigger via POST /api/admin/credit-balance/{accountId}/apply. Example: true
updated_at	TIMESTAMP NOT NULL	Last config change time
updated_by	TEXT NOT NULL	Admin user ID. Example: admin_user_42

Sample data:

operator_code	allocation_policy	overpayment_policy	overflow_wallet_ref	auto_apply
WIK	FIFO_DUE_DATE	APPLY_TO_NEXT_OPEN	NULL	true
WUG	FIFO_DUE_DATE	WALLET_TOPUP	wal_default_topup_WUG	true
WTZ	LARGEST_FIRST	APPLY_TO_NEXT_OPEN	NULL	true
YASSN	FIFO_DUE_DATE	MANUAL_REVIEW	NULL	false

How to read: Wananchi Kenya uses default policies (FIFO by due date, surplus to credit balance, auto-apply on new invoices). Wananchi Uganda directs overpayment surplus to a topup wallet instead of holding it as a credit balance. Wananchi Tanzania uses LARGEST_FIRST allocation (prioritizes paying down the largest outstanding invoices first — useful when the operator wants to minimize aging on big balances). Yas Senegal requires manual review on all overpayments and does not auto-apply credit balance (every overpayment needs an admin to decide what to do with the surplus).

API surface

Payment receipt

```
POST /api/payments/receive HTTP/1.1
Authorization: Bearer <gateway-or-admin-JWT>
Content-Type: application/json
```

```
{
  "customerId": "cust_01HX...",
  "operatorCode": "WIK",
  "paidAmount": 6402.50,
  "currency": "KES",
  "paymentMethod": "MPESA",
  "paymentReference": "mpesa_NK4XX...",
  "paidAt": "2026-05-18T10:30:00Z",
  "targetInvoiceId": null
}
```

Response 200 (full allocation):

```
{
  "success": true,
  "paymentId": "pmt_01HZ...A",
  "status": "APPLIED",
  "totalPaidAmount": 6402.50,
  "currency": "KES",
  "allocations": [
    {
      "invoiceId": "inv_01HZ...X",
      "allocatedAmount": 6402.50,
      "outstandingAfter": 0.00,
      "strategy": "AUTO_FIFO_DUE_DATE"
    }
  ],
  "totalAllocated": 6402.50,
  "totalCreditedToBalance": 0.00,
  "totalCreditedToWallet": 0.00,
  "appliedAt": "2026-05-18T10:30:05Z"
}
```

Response 200 (partial allocation with overflow to credit balance):

```
{
  "success": true,
  "paymentId": "pmt_01HZ...B",
  "status": "PARTIALLY_APPLIED",
  "totalPaidAmount": 5000.00,
  "currency": "KES",
  "allocations": [
    { "invoiceId": "inv_01HZ...Y", "allocatedAmount": 3000.00, "outstandingAfter": 0.00, "strategy": "AUTO_FIFO_DUE_DATE" },
    { "invoiceId": "inv_01HZ...Z", "allocatedAmount": 1500.00, "outstandingAfter": 1000.00, "strategy": "AUTO_FIFO_DUE_DATE" }
  ],
  "totalAllocated": 4500.00,
  "totalCreditedToBalance": 500.00,
  "totalCreditedToWallet": 0.00,
  "appliedAt": "2026-05-18T10:35:05Z"
}
```

Payment reversal

```
POST /api/payments/pmt_01HZ...D/reverse HTTP/1.1
Authorization: Bearer <admin-JWT>
Content-Type: application/json
```

```
{
  "reasonCode": "GATEWAY_CHARGEBACK",
  "reasonDetail": "M-Pesa reversed the transaction on 2026-05-18 per their reconciliation"
}

{
  "success": true,
  "reversalPaymentId": "pmt_01HZ...D_REV",
  "originalPaymentId": "pmt_01HZ...D",
  "reversedAmount": 1500.00,
  "currency": "KES",
  "reversedAllocations": [
    { "invoiceId": "inv_01HZ...Z", "reversedAmount": 1500.00, "outstandingAfter": 2500.00 }
  ],
  "reversedAt": "2026-05-18T14:00:00Z"
}
```

Admin manual surplus allocation (MANUAL_REVIEW path)

POST /api/admin/payments/pmt_01HZ...C/allocate-surplus HTTP/1.1
 Authorization: Bearer <admin-JWT>
 Content-Type: application/json

```
{
  "allocations": [
    { "targetType": "INVOICE", "targetId": "inv_01HZ...K", "amount": 1000.00 },
    { "targetType": "CREDIT_BALANCE", "amount": 1000.00 }
  ],
  "reasonCode": "MANUAL_RESOLUTION"
}
```

Admin credit balance adjustment

POST /api/admin/credit-balance/cust_01HX.../adjust HTTP/1.1
 Authorization: Bearer <admin-JWT>
 Content-Type: application/json

```
{
  "direction": "DEBIT",
  "amount": 200.00,
  "currency": "KES",
  "reasonCode": "ACCOUNTING_CORRECTION",
  "reasonDetail": "Reconciliation finding from 2026-05-15 audit"
}
```

Reads

GET /api/payments/{paymentId} — full payment ledger row with allocations.

GET /api/payments?accountId=acc_01HX_HOME&from=2026-05-01&to=2026-05-31&limit=20 — account-scoped paginated list.

GET /api/credit-balance/{accountId} — current account credit balance + recent movement.

Kafka events

Each event follows the canonical envelope:

```
{
  "eventType": "PaymentApplied",
  "aggregateId": "pmt_01HZ...A",
  "timestamp": "2026-05-18T10:30:05Z",
  "payload": { ... }
}
```

sophix.billing.payment.applied — POSTPAID invoice target

```
{
  "paymentId": "pmt_01HZ...A",
  "paymentReference": "mpesa_NK4XX...",
  "customerId": "cust_01HX...",
  "operatorCode": "WIK",
  "target": {
    "type": "INVOICE",
    "invoiceId": "inv_01HZ...X",
    "appliedAmount": 6402.50,
    "outstandingAfter": 0.00
  },
  "totalPaidAmount": 6402.50,
  "currency": "KES",
  "paymentMethod": "MPESA",
  "paidAt": "2026-05-18T10:30:00Z",
  "appliedAt": "2026-05-18T10:30:05Z"
}
```

```

    }

#### sophix.billing.payment.applied — PREPAID wallet target

```

```

{
  "paymentId": "pmt_01HZ...E",
  "paymentReference": "mpesa_NK4YY...",
  "customerId": "cust_01HY...",
  "operatorCode": "WIK",
  "target": {
    "type": "WALLET",
    "walletRef": "wal_01HZ...M",
    "appliedAmount": 3000.00,
    "balanceAfter": 5500.00
  },
  "totalPaidAmount": 3000.00,
  "currency": "KES",
  "paymentMethod": "MPESA",
  "paidAt": "2026-05-18T10:35:00Z",
  "appliedAt": "2026-05-18T10:35:02Z"
}

```

```

#### sophix.billing.payment.received-unallocated

```

```

{
  "paymentId": "pmt_01HZ...C",
  "customerId": "cust_01HX...",
  "operatorCode": "WIK",
  "paidAmount": 2000.00,
  "currency": "KES",
  "paymentMethod": "OFFLINE_RECORDED",
  "paymentReference": "manual_ref_0001",
  "receivedAt": "2026-05-18T11:00:05Z",
  "reason": "NO_OPEN_INVOICES_NO_WALLET",
  "actor": "admin_user_17"
}

```

```

#### sophix.billing.payment.reversed

```

```

{
  "paymentId": "pmt_01HZ...D_REV",
  "originalPaymentId": "pmt_01HZ...D",
  "customerId": "cust_01HY...",
  "operatorCode": "WIK",
  "reversedAmount": 1500.00,
  "reversedAllocations": [
    { "invoiceId": "inv_01HZ...Z", "amount": 1500.00 }
  ],
  "currency": "KES",
  "reversedAt": "2026-05-18T14:00:00Z",
  "reversalReasonCode": "GATEWAY_CHARGEBACK",
  "reversedBy": "admin_user_42"
}

```

Cross-DD coordination

Touched	What PAY-01 owns	What the other DD owns
BIL-01 Core	Shares the billing-charging module deployment, shares bil_01_config for module-wide settings, separate idempotency namespace from Core.	Owns charging engine.
BIL-01-CN-01 (sibling)	When CN-01 produces credit note surplus that needs to go to account_credit_balance, it calls into PAY-01's CreditBalanceService.	Owns credit/debit note application.
BIL-02-Core	Reads invoice.outstanding_amount for allocation.	Owns the invoice aggregate.
BIL-02-GEN-01	Listens for InvoiceIssued events to trigger credit-balance auto-application.	Owns invoice generation.
BIL-02-STA-01	Emits PaymentApplied / PaymentReversed; STA-01 consumes and transitions invoice status. Receives mark-paid-offline REST calls FROM STA-01.	Owns invoice status.
BIL-04 Collections	Emits PaymentReceivedUnallocated (unallocated payment may resolve an overdue situation invisible to the auto-allocator).	Owns dunning.

Touched	What PAY-01 owns	What the other DD owns
BIL-05 Wallet and Topup	Calls POST /api/wallets/credit for PREPAID payments and overflow wallet topups, POST /api/wallets/debit for reversal.	Owns wallet ops.
Payment gateway adapters	Receives POST /api/payments/receive.	Adapters own integration.
DD_NOT-01 Notification Service	Emits payment events NOT-01 consumes for customer confirmation.	Owns notification.

Dependencies

Dependency	Owner	Status
BIL-01 Charging Engine (Core)	billing team	✓ v0.9 (sibling)
BIL-02-Core	billing team	✓ v2.11
BIL-02-GEN-01	billing team	✓ v1.3
BIL-02-STA-01	billing team	✓ v1.0
BIL-04 Collections	billing team	■ pending
BIL-05 Wallet and Topup	billing team	✓ v1.0
DD_NOT-01 Notification Service	notification team	✓ v1.3
Payment gateway adapters	integrations team	■ pending per-operator
Foundation: Kafka, Auth, DB	infra team	✓

B — Current TZ

Workshop materials do not describe a unified payment application service in TZ Sophix as a separate concern. Payment receipt is likely tied into TZ's gateway-specific code paths with direct DB writes against invoice tables and wallet tables; no formal allocation policy, no credit balance entity, no payment reversal workflow.

C — Gap & bridge

PAY-01 is new functionality. It introduces:

- Unified payment receipt entry point across all gateways and admin tools
- Configurable allocation policy (FIFO by due date / by issue date / largest first / smallest first)
- Configurable overpayment policy (apply to next open / wallet topup / manual review)
- First-class account_credit_balance with auto-application on new invoices for the same account
- Formalized event contracts (PaymentApplied, PaymentReceivedUnallocated, PaymentReversed, OverpaymentPendingReview, CreditBalanceAdjusted) consumed by STA-01, BIL-04, NOT-01, analytics
- Payment reversal as a structured operation
- Full audit in payment_ledger + payment_invoice_allocation with 10-year retention